



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

Trabalho Prático Programação Orientada a Objetos

Autor / Author

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767



INSTITUTO POLITÉCNICO DE
COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, 14 dezembro de 2024

ÍNDICE

ÍNDICE	2
Introdução.....	3
Estrutura do trabalho	4
Comandos	5
1. sair	5
2. config <nome_fich>	5
3. terminar	5
4. exec <nome_fich>	5
5. precos	5
6. caravana <idCaravana>	6
7. compra <idCaravana> <quantidade>	6
8. vende <idCaravana> <quantidade>	6
9. move <idCaravana> <direcao>	6
10. auto <idCaravana>	7
12. stop <idCaravana>	7
13. barbaro <linha> <coluna>	7
14. moedas <quantidade>	8
15. tripul <idCaravana> <quantidade>	8
Itens.....	9
1. Arca do Tesouro	9
2. Caixa Pandora	9
3. Jaula	9
4. Mina	9
5. Surpresa	9
Class relevantes	9
1. Caravanas	9
2. Dados	11
3. Buffer	12
Conclusão.....	14

Introdução

O projeto "Simulador de Caravanas" tem como objetivo desenvolver um sistema que simula a movimentação de caravanas em um mapa, permitindo a leitura de dados de um arquivo, a alocação dinâmica de memória para representar o mapa e a interação com diferentes elementos do jogo, como cidades, recursos e eventos aleatórios.

Estrutura do trabalho

A estrutura deste projeto consiste em uma classe base Simulador que processa os comandos inseridos pelo utilizador, executando o mapa de jogo, lido através do ficheiro bem como o ficheiro com um conjunto de comandos predefinidos.

A classe buffer é utilizada para mostrar o estado do jogo, em paralelo criamos a classe Dados para o auxílio na gestão do jogo. O uso de variáveis e métodos estáticos permite que não seja necessário criar objetos do tipo dados para usufruir dos dados lidos do ficheiro de configuração.

A classe Posicao foi criada com o objetivo de ser mais fácil navegar pelo mapa visto que todos os objetos têm em comum coordenadas, contendo como classes derivadas as classes caravana e itens, que por sua vez são as classes base dos diferentes tipos de caravanas e itens. A vantagem de usar herança é poder usar os métodos das classes bases nas classes herdeiras.

A classe Secreta é uma subclasse da classe Caravana, projetada para representar uma caravana que pode ter apenas um tripulante. Se houver mais de um tripulante, a caravana consome uma quantidade excessiva de água (600 unidades) ao se mover, resultando em sua "morte" (remoção da lista de caravanas. O método verificar ConsumoAgua() gerência o consumo de água com base no número de tripulantes, enfatizando a necessidade de manter a caravana com apenas um tripulante para evitar a morte.

Numa tempestade tem a probabilidade de 50% de perder metade da água atual e tem 10% de probabilidade de ser destruída.

O item secreto que foi implementado é de a caravana teletransportar-se para uma cidade aleatória.

Comandos

1. sair

- Chamada: **break**;
- Ação: Encerra o programa com uma mensagem de sucesso.

2. config <nome_fich>

- Chamada: **buffer.getmapa(nome_fich)**;
 - Lê o mapa do arquivo especificado e armazena no buffer.
- Chamada: **dados.getdados(nome_fich)**;
 - Lê dados adicionais (como moedas e preços) do mesmo arquivo.
- Chamada: **buffer.displaymapa()**;
 - Exibe o mapa carregado.

3. terminar

- Chamada: **break**;
- Ação: Retorna à fase 1 do simulador, acaba a simulação.

4. exec <nome_fich>

- Chamada: **executarComandosDeArquivo(nome_fich)**;
 - Abre o arquivo e lê comandos linha por linha.
 - Para cada linha, chama **processarComando(cmd, iss)** para processar cada comando.

5. precos

- Chamada: **dados.mostrarPrecos()**;
 - Exibe os preços de compra e venda de mercadorias, além do preço da caravana.

6. caravana <idCaravana>

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID fornecido.
- Chamada: **caravana->exibirDetalhes();**
 - Exibe os detalhes da caravana encontrada.

7. compra <idCaravana> <quantidade>

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID.
- Verificação: **if (dados.getMoedas() < custoTotal)**
 - Verifica se o jogador tem moedas suficientes.
- Chamada: **caravana->setMercadoria(caravana->getMercadoria(quantidade);** +
 - Atualiza a quantidade de mercadorias na caravana.
- Chamada: **dados.setMoedas(-custoTotal);**
 - Deduz o custo das mercadorias das moedas do jogador

8. vende <idCaravana> <quantidade>

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID.
- Verificação: **if (caravana->getMercadoria() < quantidade)**
 - Verifica se a caravana tem mercadorias suficientes.
- Chamada: **caravana->setMercadoria(caravana->getMercadoria(quantidade);** -
 - Atualiza a quantidade de mercadorias na caravana.
- Chamada: **dados.setMoedas(receitaTotal);**
 - Adiciona a receita da venda às moedas do jogador.

9. move <idCaravana> <direcao>

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID.
- Chamada: **caravana->mover(direcao, buffer, dados);**

- Move a caravana na direção especificada.
- Dentro de **mover**, são feitas verificações de posição e consumo de água.
- Chamada: **buffer.displaymapa();**
 - Exibe o mapa atualizado após o movimento.

10. **auto <idCaravana>**

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID.
- Chamada: **caravana->ativarAutoGestao();**
 - Ativa o modo automático para a caravana, impedindo movimentos manuais.

12. **stop <idCaravana>**

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID.
- Chamada: **caravana->desativarAutoGestao();**
 - Desativa o modo automático da caravana, permitindo movimentos manuais.

13. **barbaro <linha> <coluna>**

- Chamada: **buffer.posicaoValida(coluna, linha);**
 - Verifica se a posição especificada é válida.
- Chamada: **Barbara *novaBarbara = new Barbara(dados);**
 - Cria uma nova caravana bárbara.

- Chamada: **novaBarbara->setPosicao(coluna, linha);**
 - Define a posição da nova caravana.
- Chamada: **buffer.atualizarPosicaoCaravana(novaBarbara->getId(), coluna, linha, true);**
 - Atualiza o mapa com a nova posição da caravana bárbara.

14. moedas <quantidade>

- Chamada: **dados.setMoedas(teste);**
 - Atualiza a quantidade de moedas do jogador.
- Verificação: Se o valor não for fornecido, exibe um erro.

15. tripul <idCaravana> <quantidade>

- Chamada: **Caravana::encontrarCaravanaPorId(idCaravana);**
 - Busca a caravana pelo ID.
- Verificação: **if (dados.getMoedas() < custoTotal)**
 - Verifica se o jogador tem moedas suficientes para comprar os tripulantes.
- Chamada: **caravana->setTripulacao(caravana->getTripulacao() + quantidade);**
 - Atualiza a quantidade de tripulantes na caravana.
- Chamada: **dados.setMoedas(-custoTotal);**
 - Deduz o custo dos tripulantes das moedas do jogador.

Itens

1. Arca do Tesouro

- **Resumo:** A Arca do Tesouro é um item que, quando encontrado pela caravana, aumenta a quantidade de moedas disponíveis.
- **Função:** `void ArcaTesouro::aplicarEfeito(Caravana &caravana)` - Esta função calcula 10% a mais de moedas com base nas moedas atuais e atualiza o total de moedas da caravana.

2. Caixa Pandora

- **Resumo:** A Caixa Pandora é um item que reduz a tripulação da caravana em 20% quando encontrada.
- **Função:** `void CaixaPandora::aplicarEfeito(Caravana &caravana)` - Esta função diminui a quantidade de tripulantes da caravana em 20%.

3. Jaula

- **Resumo:** A Jaula é um item que permite à caravana capturar prisioneiros, aumentando assim o número de tripulantes.
- **Função:** `void Jaula::aplicarEfeito(Caravana &caravana)` - Esta função gera um número aleatório de prisioneiros (até 40) e os adiciona à tripulação da caravana.

4. Mina

- **Resumo:** A Mina é um item que permite à caravana extrair recursos, embora a implementação específica do efeito não esteja detalhada.
- **Função:** `void Mina::aplicarEfeito(Caravana &caravana)` - Esta função chama o método `mina()` da classe **Caravana**, que deve conter a lógica para a extração de recursos.

5. Surpresa

- **Resumo:** A Surpresa é um item que leva a caravana a uma cidade aleatória, possivelmente oferecendo novas oportunidades ou desafios.
- **Função:** `void Surpresa::aplicarEfeito(Caravana &caravana)` - Esta função chama o método `cidadeAleatoria()` da classe **Caravana**, que deve conter a lógica para mover a caravana para uma cidade aleatória.

Class relevantes

1. Caravanas

1. **Atributos:**

- **Identificação:** Cada caravana possui um **id** único, que é gerado automaticamente.
- **Posição:** A posição da caravana é representada por coordenadas **x** e **y**.
- **Recursos:** A caravana possui atributos para **mercadoria**, **água** e **tripulação**, que representam os recursos disponíveis.
- **Auto-gestão:** Um atributo booleano **autoGestao** indica se a caravana está em modo automático, o que impede movimentos manuais.

2. Construtores:

- A classe possui um construtor padrão e um construtor que permite inicializar a caravana com valores específicos para mercadoria, água, tripulação e posição.

3. Métodos de Acesso:

- Métodos **get** e **set** permitem acessar e modificar os atributos da caravana.

4. Gestão de Movimento:

- O método **mover** permite mover a caravana em direções específicas (cima, baixo, esquerda, direita, etc.), desde que não esteja em modo automático. Ele verifica se a nova posição é válida usando um objeto **Buffer** e atualiza a posição da caravana no mapa.

5. Auto-gestão:

- Métodos para ativar e desativar o modo automático, bem como verificar se a caravana está nesse modo.

6. Verificação de Consumo de Água:

- Um método virtual **verificarConsumoAgua()** que deve ser implementado em classes derivadas para gerenciar o consumo de água da caravana.

7. Interação com o Mapa:

- O método **cidadeAleatoria** permite que a caravana se mova para uma posição aleatória no mapa, desde que essa posição seja válida (representada por um caractere minúsculo).

8. Gerenciamento de Caravanas:

- A classe mantém um vetor estático **todasAsCaravanas** que armazena todas as instâncias de caravanas criadas, permitindo a busca por ID e a remoção de caravanas quando necessário (como no método **mina**).

9. Exibição de Detalhes:

- O método **exibirDetalhes** imprime informações sobre a caravana, incluindo ID, posição, mercadoria, água e tripulação.

A classe **Caravana** encapsula a lógica e os dados necessários para gerenciar uma caravana em um jogo, permitindo movimentação, gestão de recursos e interação com o ambiente.

2. Dados

1. Atributos:

- **Dimensões do Mapa:** A classe possui atributos estáticos **colunas** e **linhas** que definem as dimensões do mapa do jogo.
- **Recursos:** Atributos como **moedas**, **duracao_item**, **max_itens**, **preco_venda_mercadoria**, **preco_compra_mercadoria**, e **preco_caravana** armazenam informações sobre os recursos e preços no jogo.
- **Cidades:** Um vetor **cidades** armazena instâncias da classe **Cidade**, representando as cidades no mapa.
- **Mapa:** Um ponteiro duplo **mapa** é utilizado para representar o mapa do jogo, onde cada célula pode conter informações sobre o terreno ou entidades.

2. Construtor e Destrutor:

- O construtor inicializa as dimensões do mapa e aloca memória para o mapa.
- O destrutor libera a memória alocada para o mapa, evitando vazamentos de memória.

3. Métodos de Acesso:

- Métodos **get** e **set** permitem acessar e modificar os atributos da classe, como as dimensões do mapa e o número de moedas.

4. Leitura de Dados:

- O método **getdados** lê as configurações do jogo a partir de um arquivo, incluindo dimensões do mapa, preços e outros parâmetros. Ele também chama **criarCaravanas** para instanciar as caravanas com base nos dados do mapa.

5. **Criação de Caravanas:**

- O método **criarCaravanas** percorre o mapa e cria instâncias de diferentes tipos de caravanas (militar, comércio, secreta) com base nos caracteres encontrados no mapa. As caravanas são automaticamente adicionadas ao vetor estático na classe **Caravana**.

6. **Criação de Cidades:**

- O método **criarCidades** percorre o mapa e cria instâncias da classe **Cidade** para cada letra minúscula encontrada, representando uma cidade no mapa.

7. **Exibição de Preços:**

- O método **mostrarPrecos** exibe os preços das mercadorias e da caravana, formatando a saída em uma string antes de imprimi-la.

8. **Métodos Estáticos:**

- Métodos estáticos como **getColunas**, **getLinhas**, **getMoedas**, **getduracao_tempo**, **getmax_itens**, e **getpmapa** permitem acessar informações globais sobre o estado do jogo sem a necessidade de instanciar a classe.

A classe **Dados** centraliza a gestão das informações do jogo, incluindo a configuração do mapa, a criação de caravanas e cidades, e a manipulação de recursos. Ela é fundamental para a inicialização e o funcionamento do jogo, fornecendo uma interface para aceder e modificar dados essenciais. A classe também garante que a memória seja gerida corretamente, evitando vazamentos.

3. Buffer

1. **Atributos:**

- **Dimensões do Mapa:** A classe possui atributos **colunas** e **linhas** que definem as dimensões do mapa.

- **Mapa:** Um ponteiro duplo **mapa** é utilizado para representar o mapa do jogo, onde cada célula pode conter informações sobre o terreno ou entidades (como caravanas).

2. Construtor e Destrutor:

- O construtor inicializa as dimensões do mapa e aloca memória para o mapa, criando uma matriz de caracteres.
- O destrutor libera a memória alocada para o mapa, evitando vazamentos de memória.

3. Métodos de Acesso:

- Métodos **getColunas** e **getLinhas** permitem acessar as dimensões do mapa.
- Métodos **setColunas** e **setLinhas** permitem modificar as dimensões do mapa.

4. Exibição do Mapa:

- O método **displaymapa** retorna uma representação em string do mapa, permitindo que o estado atual do mapa seja visualizado.

5. Leitura do Mapa:

- O método **getmapa** lê a configuração do mapa a partir de um arquivo, alocando memória conforme necessário e preenchendo a matriz **mapa** com os dados lidos.

6. Limpeza do Mapa:

- O método **limpamapa** redefine todas as células do mapa para um espaço em branco, efetivamente limpando o mapa.

7. Validação de Posições:

- O método **posicaoValida** verifica se as coordenadas fornecidas estão dentro dos limites do mapa e se a posição não é ocupada por um obstáculo (como montanhas ou outras entidades).

8. Atualização da Posição da Caravana:

- O método **atualizarPosicaoCaravana** atualiza a posição de uma caravana no mapa. Ele limpa a posição anterior da caravana e define a nova posição, utilizando diferentes símbolos para identificar caravanas normais e bárbaras.

A classe **Buffer** é fundamental para a gestão do mapa do jogo, permitindo a leitura, exibição e atualização do estado do mapa. Ela garante que a memória seja gerida corretamente e fornece métodos para validar movimentos e atualizar a posição das caravanas. A classe atua como uma interface entre a lógica do jogo e a representação visual do mapa, facilitando a interação entre as diferentes entidades do jogo.

Conclusão

Para concluir, o projeto "Simulador de Caravanas" foi um trabalho interessante. Embora não tenhamos conseguido implementar tudo o que estava no enunciado, conseguimos criar um sistema que funciona e que simula a movimentação das caravanas, a interação com as cidades e a gestão de recursos.